TP Sécurité IoT
Architecture et Sécurité des Réseaux sans Fil
LoRa, Authentification, Confidentialité, Intégrité,
LoRaWAN, OTAA, ABP.

INSA-Lyon
Département Télécom
Walid Bechkit

Objectifs : Cette séance de travaux pratiques a principalement quatre objectifs :

- La mise en place et la compréhension d'une simple communication LoRa entre un émetteur et un récepteur (prise en main du protocole radio LoRa).
- Compréhension et mise en place des trois services de base en sécurité : l'**authentification**, l'**intégrité** et la **confidentialité**.
- Compréhension et prise en main de l'architecture LoRaWAN et le protocole LoRaWAN associé.
- Compréhension et mise en place des modes de sécurité LoRawAN OTAA et ABP.

Mots clés : *MKR WAN 1310, LoRa, contrôle de puissance, spreading factor, coding rate, CRC, services de base de sécurité, authentification, intégrité, confidentialité, LoRaWAN, Device, Gateway, Serveur Réseau, Serveur Application, Format de trame LoRaWAN, TTN, OTAA, ABP, Hash, MIC, etc.*

Préambule : Carte MKR WAN 1310

Durant ce TP, nous utiliserons des cartes Arduino MKR WAN 1310. La carte Arduino MKR WAN 1310 offre une connectivité LoRa/LoRaWAN à faible consommation d'énergie. Elle intègre entre autres un processeur à faible consommation d'énergie Microchip SAMD21, le module Murata CMWX1ZZABZ LoRa, une crypto chip (ECC508), et une mémoire flash de 2 Mo.

Commencer par télécharger la dernière version d'Arduino pour Linux 64 bits sur le site web Arduino. Brancher vos cartes MKR 1310, **lancer Arduino en root** et si cela n'est pas fait, installer le paquet Arduino SAMD Boards (Cortex-M0+) qui permet de prendre en charge la famille des cartes MKR entre autres. Installer enfin la dernière version de la bibliothèque LoRa "by Sandeep Mistry" (pour les parties 1 et 2) et MKRWAN (pour les parties 3 et 4).

1 LoRa : prise en main et compréhension des paramètres radio



FIGURE 1 – Architecture des parties 1 et 2 : simple communication LoRa entre un émetteur et un récepteur

Durant les deux premières parties de ce TP, nous allons principalement s'intéresser au protocole radio LoRa et implémenter les services de sécurité de base entre un émetteur et un récepteur. Pour cela nous allons mettre en place une simple architecture avec un émetteur et un récepteur communicant avec le protocole LoRa. Afin de mettre en oeuvre une communication LoRa, nous allons utiliser la bibliothèque LoRa que vous venez d'installer. Cette bibliothèque est largement utilisée et elle est compatible avec les MKRWAN 1300 et 1310 ainsi que les cartes basées sur des modules LoRa Semtech SX1276/77/78/79 (Dragino Lora Shield, HopeRF RFM95W, RFM96W, RFM98W , etc.). D'autres bibliothèque plus riches existent comme RH_RF95.h compatible avec les module Grove LoRa.

Télécharger les deux fichiers du TP : TX.ino et RX.ino. Lire attentivement et comprendre ces programmes qui assurent l'émission et la réception LoRa.

1.1 Choix de la fréquence et premier test

La fonction *void LoRa.begin(frequency)* permet d'initialiser le module Lora sur la fréquence porteuse frequency. Au niveau des programmes TX et RX, mettre à jour la fréquence centrale (porteuse). Afin de réduire les interférences entre les différents groupes, prenons aléatoirement une des fréquences suivantes : 867.3 Mhz, 867.5 Mhz, 867.7 Mhz, 867.9 Mhz et 868.1 Mhz. Charger ensuite le programme TX dans un noeud Arduino et le programme Rx dans un autre. Qu'observe-t-on ? les messages sont ils bien reçus ?

La fonction "setTxPower" permet de fixer la puissance du niveau d'émission. Les valeurs de txPower peuvent prendre les valeurs de 2 à 20 dBm.

```
LoRa.setTxPower(txPower);
```

Pour la suite de ce TP, fixons les puissances d'émission à 10dBm = 10 mW.

Les étudiants qui ne suivront pas de cours approfondi sur LoRa/LoRaWAN et qui souhaitent approfondir la compréhension des paramètres de la couche physique LoRa, peuvent mener l'évaluation de performances du débit effectif proposées dans les deux sous sections suivantes. Pour les autres, vous pouvez directement passer à la section "structure du message physique LoRa"

1.2 [Bonus] Mesure du temps de transmission

La fonction *unsigned long micros()* retourne le temps actuel en micro-secondes. Dans le programme TX, sauvegarder le temps courant dans la variable "Temps_debut_tx" juste avant l'appel à la fonction Lora.endPacket qui lance l'émission d'un paquet. Calculer juste après cette fonction le temps que la transmission a pris en millisecondes dans la variable Temps_total_tx. Afficher ensuite ce temps avec les message de succès ou d'échec.

Exemples :

Transmission du paquet : 10... Succes ... Temps de transmission : 36.54 ms

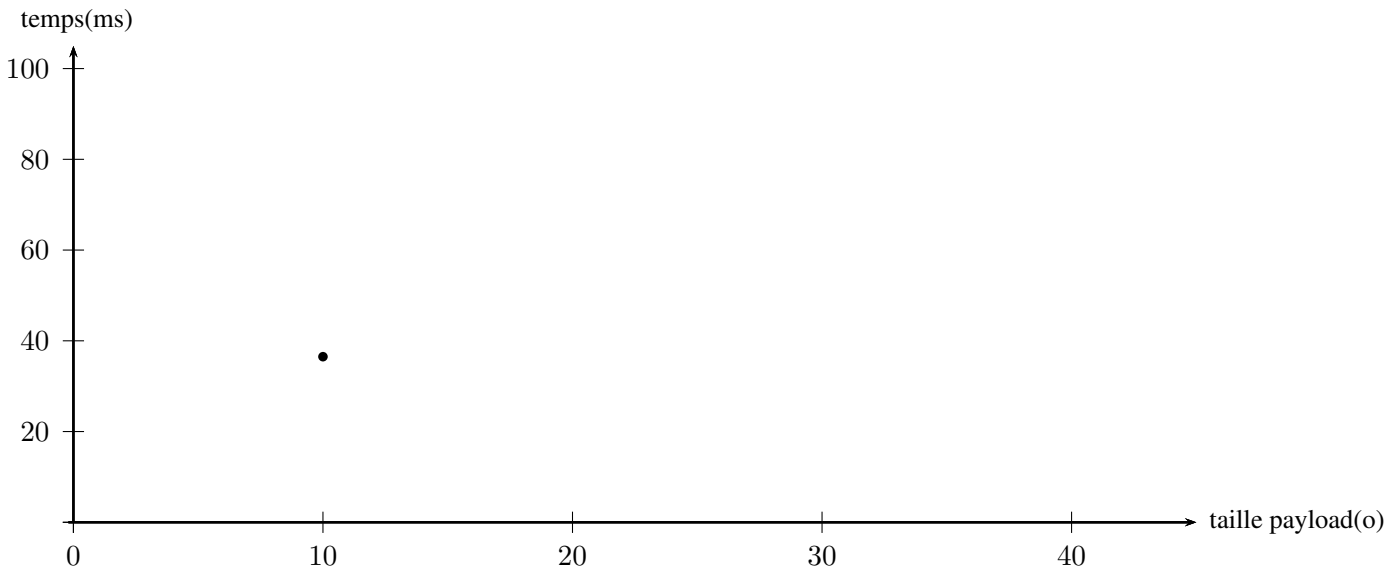
Transmission du paquet : 11... Echec ... après : 100.2 ms

Une fois le programme TX.ino mis à jour, lancer le moniteur Série et visualiser les résultats.

Ce temps suffira-t-il selon vous pour calculer le débit de transmission en bits/s ? Si oui calculer ce débit, sinon pourquoi ?

Changer maintenant la taille de paquet à envoyer en octets (la variable "nb_octets_a_envoyer") en testant les valeurs 10, 20, 30 et 40 octets. Vérifier à chaque fois le message reçu et noter le temps de transmission dans le tableau ci-dessous.

Taille des paquets	10	20	30	40
Temps				



Estimer maintenant le débit effectif pour cette configuration par défaut.

1.3 [Bonus] Paramètres de la couche physique LoRa

1.3.1 Facteur d'étalement de spectre

```
LoRa.setSpreadingFactor(spreadingFactor);
```

La fonction ci-dessus permet de positionner la valeur du SF entre 6 et 12 (valeur par défaut égale à 7).

1.3.2 Largeur de canal (bandwidh)

La largeur du canal autour de la fréquence centrale est un autre paramètres très important qui a un impact sur le débit de transmission.

```
LoRa.setSignalBandwidth(signalBandwidth);
```

La fonction ci-dessus permet de fixer la valeur de la largeur du canal qui peut prendre les valeurs 7.8E3, 10.4E3, 15.6E3, 20.8E3, 31.25E3, 41.7E3, 62.5E3, 125E3 (valeur par défaut), et 250E3. Afin d'étudier l'impact de ce paramètre, nous allons essayer (en fixant le SF à 8 par exemple) les largeurs 62.5E3 (62.5 khz), 125E3 (125khz qui est la valeur par défaut), et 250E3 (khz).

1.3.3 Coding Rate C/R

Le coding rate est un paramètre très important afin de détecter et de corriger les erreurs de transmission. les valeurs possibles du C/R avec LoRa sont 4/5 4/6, 4/7 et 4/8. Un C/R de 4/8 signifie que nous envoyons deux fois plus de bits que le message initial.

```
LoRa.setCodingRate4(codingRateDenominator);
```

L'instruction ci-dessus permet de fixer le dénominateur du C/R qui peut prendre des valeurs 5 (valeur par défaut),6,7 ou 8. (le numérateur étant fixé à 4).

1.4 Structure du message physique LoRa

LoRa propose deux modes de format de paquet :

- Le mode d'en-tête explicite comprend un en-tête court qui contient des informations sur la longueur du payload, le C/R et la présence ou pas d'un CRC. Ce mode là est le mode utilisé par défaut et c'est celui que nous avons utilisé jusque-là.

Préambule (8s)	Entete Physique (0-3o) Longueur ; C/R ; CRC ?	Payload (max = 255o)	CRC (0-2o)

- Le mode d'en-tête implicite dans lequel la taille du payload, le taux de codage et la présence du CRC sont fixes. Dans ce cas, ces informations ne sont pas envoyées ce qui permet de réduire la taille des données envoyées. Néanmoins, cela nécessite la configuration de ces paramètres manuellement des deux cotées.

Préambule (8s)	Payload (max = 255o)	CRC (0-2o)

La bibliothèque utilisée pendant ce TP utilise par défaut le mode explicite. Néanmoins tout paquet peut être envoyé en mode implicite (sans entêtes de la couche physique) grâce à la commande :

```
LoRa.beginPacket(true);
```

Nous gardons dans ce qui suit l'utilisation du mode explicite sans modifier nos programmes. Néanmoins si le temps le permet, vous pouvez tester le mode explicite en mettant à jour les programmes des deux cotés.

L'utilisation du CRC est par défaut désactivée dans la bibliothèque que nous utilisons durant ce TP. Les deux instructions suivantes permettent d'activer (respectivement désactiver) l'utilisation du CRC.

```
LoRa.enableCrc();
LoRa.disableCrc();
```

1.5 Formatage du payload

Comme vous l'avez remarqué, les messages physiques de LoRa ne permettent pas d'identifier les sources et destinataires des messages physiques envoyés ni de gérer l'accès multiple à un canal de transmission. Afin d'introduire LoRaWAN, commençons par définir nous même une structure de trame de niveau 2 qui contient l'adresse du noeud source, l'adresse du noeud destinataire, un numéro de trame et les données utiles de température et d'humidité. Afin de simplifier la tache, nous définissions des adresses de noeuds sur un seul octet.
Dans les deux fichiers TX et RX, déclarer tout au début de votre fichier (idéalement après les include ou dans un autre fichier d'en-tête que vous devrez inclure) la structure suivante :

```
typedef struct
{
  byte        destination;
  byte        source;
  int         num_trame;
  float       temperature;
  float       humidite;
} Trame_t;
```

Au niveau de l'émetteur, déclarer une variable de type Trame_t (ex. Trame_t ma_trame). Juste avant l'envoi du paquet, remplir les différents champs de cette structure (ex. ma_trame.temperature= 25.76), et remplacer l'instruction LoRa.write() précédente par :

```
LoRa.write( (byte* )& ma_trame, sizeof(ma_trame) );
```

où ma_trame est votre structure qui contient le paquet à envoyer.

Au niveau du récepteur, déclarer une variable de type Trame_t (ex. Trame_t ma_trame). Remplacer ensuite la boucle qui lit les octets reçus un par un par :

```
LoRa.readBytes( (byte*)& ma_trame, sizeof(ma_trame));
```

Afficher les champs de la structure reçue. S'assurer que toutes informations sont bien reçues.

2 Confidentialité, authentification, et intégrité

Durant cette partie, seules des communications LoRa seront mis en place (bibliothèque LoRa) et nous implémenterons nous même des services d'authentification, d'intégrité et de confidentialité à travers un chiffrement symétrique AES-128. Les implémentations prévues sont volontairement similaires à la norme LoRaWAN afin de bien comprendre la conception des solutions LoRaWAN.

2.1 Authentification

L'objectif de cette partie est de compléter un simple échange entre entre un client et un serveur pour comprendre l'authentification.

- o Commencer par télécharger les programmes fournis.
- o Le formateur vous expliquera le protocole d'authentification imaginé. Ce protocole est très simple et reprend approximativement la logique des échanges prévus dans le protocoles LoRaWAN.
- o Lire les programmes et compléter les quelques lignes manquantes dans le programme du serveur.

Tester les programmes client et serveur et valider le comportement de l'authentification.

Changer un seul bit dans la clé utilisée au niveau du serveur. Relancer le client pour valider bien l'échec de l'authentification dans ce cas.

Observer dans ce cas l'impact sur le MIC envoyé.

Est ce que ces échanges permettent de garantir l'authentification du serveur auprès du client ? C'est quoi le risque dans ce cas ? Sans l'implémenter, proposer une extension permettant de résoudre ce problème.

2.2 Génération de clés

Dé-commenter la partie dans laquelle la clé de session du chiffrement est générée et installée des deux cotées : client et serveur. En s'inspirant de cette partie, ajouter une autre qui permet de générer et d'installer la clé d'intégrité en suivant le protocole qu'on vous expliqué au départ.

2.3 Confidentialité et Intégrité

Afin de comprendre l'indépendance des deux fonctionnalités, nous allons mettre en, place des transmission de données : i) non chiffrées et sans intégrité, ii) chiffrées et sans intégrité, iii) non chiffrées avec code d'intégrité et enfin chiffrées avec code d'intégrité.

Dé-commenter la partie dans la quelle des données sont envoyées. Cet envoi correspond à quel scénario ?

En s'inspirant de cette partie, compléter les programmes afin de permettre d'envoyer des données avec les trois autres scénarios cités ci-dessus.

Au niveau du serveur,compléter le programme afin de permettent de vérifier le MIC à la réception dans les deux scénarios avec intégrité. Tester ces programmes.

Au niveau du serveur,compléter le programme afin de permettent de déchiffrer les messages chiffrées reçues dans les deux scénarios où les données sont chiffrées. Tester ces programmes.

3 Introduction à LoRaWAN : Architectures et implémentations

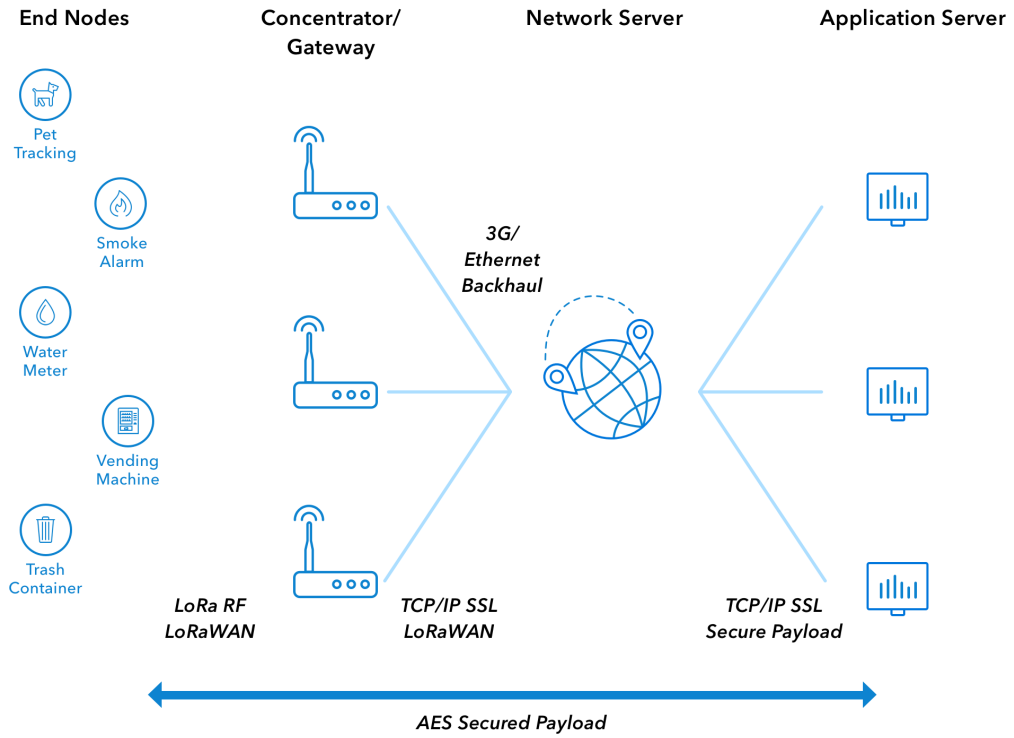


FIGURE 2 – Architecture d'un réseau loRaWAN (source : www.thethingsnetwork.org)

3.1 Architecture globale

Comme illustré en Figure 2, l'architecture des réseaux LoRaWAN repose sur quatre éléments :

- **Noeud (Device LoRaWaN)** : Objet d'extrémité qui a comme principal rôle la transmission des données (souvent issues de capteurs) en voix montante vers le réseau. Selon le besoin, ces noeuds peuvent aussi recevoir des données en voix descendante ce qui les distingue en trois classes (A,B et C).
- **Passerelle (Gateway LoRaWAN)** : Équipement qui permet de faire le lien entre les devices et le réseau coeur LoRaWaN (Serveurs réseau et application). Elle constitue le pont entre les appareils et le réseau The Things Network. Elle communique avec les objets en utilisant le protocole LoRa à basse consommation d'énergie, alors qu'elle est souvent connectée au réseau coeur grâce à des protocoles à haut débit comme le WiFi, Ethernet ou des réseaux 4G/5G. Une seule passerelle peut desservir des milliers de noeuds.
- **Serveur réseau :** Le serveur réseau reçoit tous les messages en provenances des Gateway connectés à lui. Son rôle sera entre autres de i) vérifier l'intégrité des données reçues, ii) de dé-dupliquer les messages (détails en TP4), iii) faire le lien avec le serveur applicatif, iv) sélectionner la meilleure passerelle pour transmettre les messages en voix descendante.
- **Serveur applicatif :** Son rôle est de déchiffrer les messages et les présenter aux utilisateurs. Le post-traitement, stockage et analyse de ces données peuvent être intégrés au serveur applicatif mais sont hors standard LoRaWAN.

3.2 Identifiants et clés importants à connaître

Les principaux identifiants à connaître pour comprendre et pouvoir mettre en place un réseau LoRaWAN sont les suivants :

- **DevEUI** : 64 bit identifiant physique du noeuds (device), EUI-64, unique mondialement, attribué à la fabrication
- **DevAddr** : 32 bit adresse du nud : dynamique, non unique, attribuée grâce à la procédure dactivation.
- **GatewayEUI** : 64 bit identifiant physique de la passerelle, EUI-64 (unique)
- **AppEUI** : 64 bits identifiant de lapplication, EUI-64, unique par réseau

Par ailleurs les clés utilisées et qui sont aussi très importantes à connaitre sont les suivantes (détails lors du prochain TP) :

- **AppKey** Clé primaire utilisée pour dériver les clés de sessions
- **AppSKey** Clé de session de confidentialité (chiffrement des données)
- **NwkSKey** clé de session dintégrité des données

3.3 Différentes implémentations des réseaux LoRaWan

3.3.1 Réseaux opérés

Les opérateurs se lancent activement sur des déploiement de réseaux LoRaWAN. En France Orange et Objenious (Bouyges) déploient et opèrent des réseaux d'envergure avec une couverture importante. Les parties : "passerelles", "serveurs réseaux" et "serveurs applicatifs" sont gérées par les opérateurs et l'utilisateur n'a qu'à gérer les noeuds d'extrémité. Généralement une interface web et/ou une application est proposé pour configurer les devices, les applications, etc.

3.3.2 Réseau privés

Chaque entreprise, collectivité ou particulier peut développer et mettre en place facilement son propre réseau LoRaWAN. On peut configurer des passerelles LoRaWAN de différents types (simple passerelle à base de Raspberry Pi et du module RHF0M301 au passerelles plus professionnelles comme les LORIX One). Ensuite, il suffit d'installer un serveur réseau et un serveur applicatif sur des simples machines ou sur des serveurs plus puissants. Plusieurs version de serveurs LoRaWAN sont disponibles en open source.

3.3.3 Réseaux Communautaires

Plusieurs réseaux Communautaires se mettent en place dans le monde dont le plus important est TTN (The Things Network). TTN est un réseau LoRaWAN communautaire et open source qui compte aujourd'hui plus de 40000 contributeurs de 90 pays (4000 passerelles).

3.4 Noeuds (Device LoRaWAN) : initialisation et identification du DevEUI

Comme expliqué précédemment, nous allons utiliser durant ce TP la bibliothèque MKRWAN qui implémente le protocole LoRaWAN. Commencer par installer cette bibliothèque si elle ne l'est pas (Tools –> Manage Librairies). Télécharger le code fourni (Partie 3_4)$quipermettoutsimplementd'initialiserlemoduleLoraWANetd'affic$

L'instruction begin permet d'initialiser le module LoRaWAN en prenant en paramètre la bande de fréquence parmi les suivante : AS923, AU915, EU868 (default value), KR920, IN865, US915 et US915_HYBRID.

```
modem.begin(band);
```

Faudrait-il changer la valeur utilisée dans les programmes fournis ? Si oui modifiez la et rechargez les programmes. Noter les valeurs "device EUI" de vos deux capteurs.

Capteurs	Capteur 1 Groupe ..	Capteur 2 Groupe ..
Device EUI	A8 61 0A ..	A8 61 0A ..

3.5 Configuration des devices et leurs enregistrement auprès du TTN

Commencer donc par créer un compte sur TTN (chaque participant ou chaque binôme). Pour cela, se connecter sur https ://www.thethingsnetwork.org/https ://www.thethingsnetwork.org/ et remplir le formulaire sur "Sign Up".

Créer maintenant une nouvelle application sur TTN, et enregistrer maintenant votre noeud capteur (device) auprès du serveur applicatif TTN dans l'application créée. Pour cela, vous avez besoin de votre DevEUI. Notez que de Device ID demandé ne sera jamais échangé sur le réseau LoRaWAN et n'est utilisé que par le serveur applicatif.

Analysez les résultats affichés sur le moniteur série du MKR d'un coté et sur la passerelle et sur l'application d'un autre coté. Qu'observe-t-on ?

Comme vu en cours, une fois les noeuds capteurs authentifiés et connectés au réseau, ce dernier leur attribue un DevAddr.

```
modem.getDevAddr();
```

Grâce à la commande ci-dessus, affichez et notez les DevAddr de vos deux noeuds capteurs. Ces derniers seront-elles toujours les mêmes si on redemande une connexion à ce réseau après quelques heures ?

Les messages envoyés par les programmes proposés ("bienvenue au cours de securite IoT") sont des simples messages texte (codage ASCII). Nous pouvons évidement envoyer d'autres types de données comme nous l'avons fait précédemment.

Afin de vérifier que le message envoyé est bien reçu par l'application, copier **au niveau de l'application**, le payload reçu et le décoder sur n'importe quel decodeur base64. Qu'observe-t-on ? le message transmis est-t-il bien reçu ?

Refaire les mêmes étapes **au niveau d'une passerelle** si vous avez accès à une passerelle TTN. Qu'observe-t-on ? pouvons nous décoder les payload au niveau de la passerelle ? pourquoi ?

4 Sécurité LoRaWAN

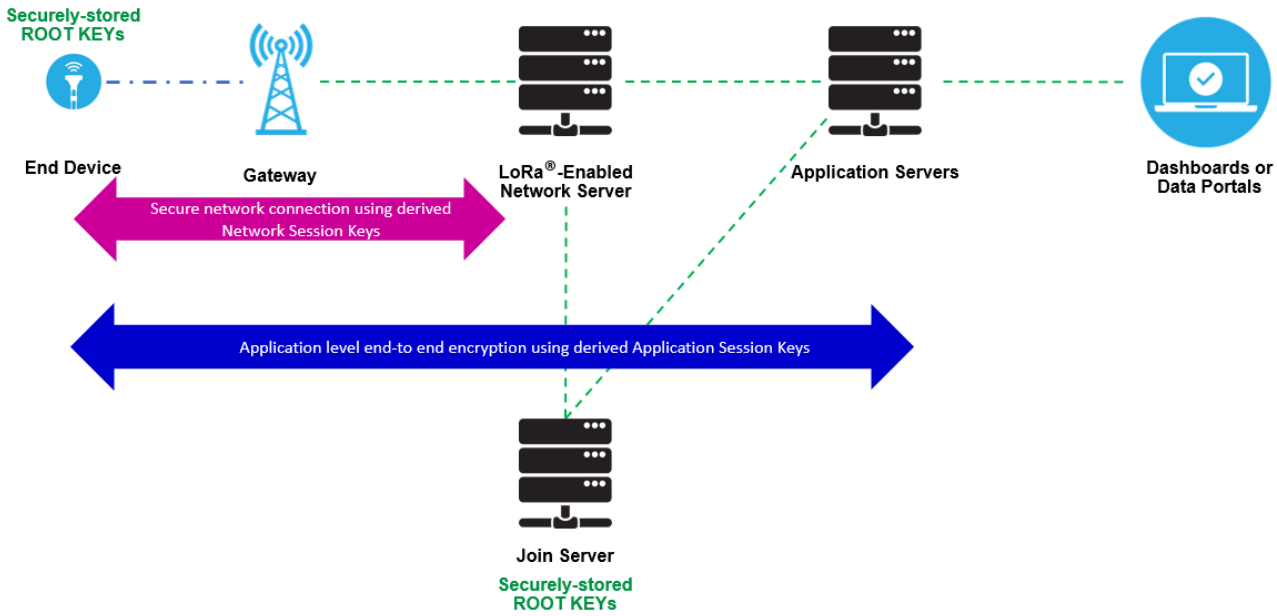


FIGURE 3 – Architecture de sécurité LoRaWAN (source : https ://lora-developers.semtech.com/library/tech-papers-and-guides/lora-and-lorawan/)

LoRaWAN fonctionne par défaut en mode OTTA. Relancer les capteurs et vérifier que les données sont bien reçues au niveau de votre application.

4.1 Mode OTTA : Over The Air Activation

```
String getAppSKey()
String getNwkSKey();
```

Afficher les clés deux session (AppSKey et NwkSKey) des deux cotées : au niveau des capteurs (fonctions ci-dessus) et niveau des serveurs applicatifs. Vérifier que les clés sont à chaque fois identiques.

Sur la base de la première partie et du cours, expliquer le rôle de ces deux clés et les entités où elles sont utilisées.

4.2 Mode ABP : Activation By Personnalization

Commencer par supprimer les noeuds enregistrés sur TTN et les enregistrer à nouveau en mode ABP. Afin d'activer les noeuds en mode ABP, la bibliothèque MKRWAN propose l'instruction suivante :

```
int joinABP(const char * devAddr, const char * nwkSKey, const char * appSKey)
```

Pourquoi cette activation nécessite de préciser le devAddr en plus des deux clés de sessions ?

Enregistrer maintenant les noeuds en utilisant le mode ABP. Mettre à jour les programmes Arduino pour que cela fonctionne. Tester ce mode et commenter.

N'oubliez svp pas de dés-enregistrer toutes les Gateways et Devices LoRaWAN utilisés lors de ce TP. Voila ! C'est fini. Merci beaucoup et très bonne continuation

Références

— The Things Network : https ://www.thethingsnetwork.org/https ://www.thethingsnetwork.org/

— Jansen C. Liando, Amalinda Gamage, Agustinus W. Tengourtius, and Mo Li. Known and Unknown Facts of LoRa : Experiences from a Large-scale Measurement Study. ACM Trans. Sen. Netw. 15, 2, 2019

— Dedicated networks for IoT : PHY / MAC state of the art and challenges C. Goursaud, JM. Gorce, EAI Endorsed Transactions on the Internet of Things, 2016

— Cours Fomation LoRa / LoRaWAN, Sylvain MONTAGNY, Université Savoie Mont Blanc

— Caractérisation des procédés de modulation et de démodulation LoRa, J-F SERREAU, 2019

— Projet IMA4 P6 : LoRaWAN, Gosse Antoine 2018. https ://projets-ima.plil.fr/mediawiki/images/f/f6/Rapport_P6_LoR
_Gosse_Antoine.pdf

— https ://wiki.seeedstudio.com/LoRa_LoRaWan_Gateway_Kit/

— https ://lora.readthedocs.io/en/latest/

— https ://get2hub1.blogspot.com/2020/04/available-iot-network-technology.html.

— Documentation de la bibliothèque LoRa, Sandeep Mistry, disponible en ligne sur l'adresse :
https ://github.com/sandeepmistry/arduino-LoRa/blob/master/API.md

— Caractérisation des procédés de modulation et de démodulation LoRa, J-F SERREAU, 2019

— http ://www.onditech.com/documents/freq_433_868.pdf

— https ://www.mobilefish.com/download/lora/lora_part17.pdf